ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

BÁO CÁO
ĐỒ ÁN THIẾT KẾ II

ĐỀ TÀI:

THIẾT KẾ HỆ THỐNG ĐO NHIỆT ĐỘ VÀ CẢNH BÁO NGƯỠNG SỬ DỤNG ATMEGA16

Giảng viên hướng dẫn:	ThS. Vũ Hồng Vinh	
Sinh viên thực hiện:	Mai Duy Hiếu	20223967
	Hoàng Quốc Hưng	20233758
Mã lớp học:	772642	
Nhóm thực hiện:	Nhóm 4	

Hà Nội, 11/2026

LỜI NÓI ĐẦU

Trong bối cảnh phát triển mạnh mẽ của khoa học kỹ thuật và cuộc cách mạng công nghiệp 4.0, các hệ thống nhúng thông minh, đo lường tự động và giám sát môi trường ngày càng đóng vai trò thiết yếu trong đời sống và sản xuất. Đặc biệt, thông số nhiệt độ và độ ẩm là hai yếu tố vật lý cơ bản, ảnh hưởng trực tiếp đến sự vận hành ổn định của hệ thống máy chủ dữ liệu (Data Center), quy trình ấp nở và bảo quản nông sản, nhà kính nông nghiệp công nghệ cao, cũng như an toàn lao động trong nhà xưởng công nghiệp. Việc xây dựng một hệ thống giám sát nhiệt độ tin cậy, có khả năng hiển thị thời gian thực và tự động đưa ra cảnh báo khi vượt ngưỡng cho phép là bài toán kỹ thuật mang tính thực tiễn rất cao.

Học phần Đồ án Thiết kế II là cơ hội quan trọng giúp sinh viên ngành Kỹ thuật Điện tử – Viễn thông củng cố, liên kết các kiến thức nền tảng về cấu trúc máy tính, kỹ thuật vi xử lý, kỹ thuật lập trình C nhúng và thiết kế giao tiếp ngoại vi để giải quyết một bài toán kỹ thuật trọn vẹn từ thiết kế sơ đồ, lập trình phần mềm cho tới thử nghiệm trên kit thực tế.

Đề tài "Thiết kế hệ thống đo nhiệt độ và cảnh báo ngưỡng sử dụng ATmega16" được thực hiện nhằm xây dựng một mô hình đo lường số hoàn chỉnh. Hệ thống sử dụng vi điều khiển AVR ATmega16 làm trung tâm xử lý, giao tiếp trực tiếp với cảm biến số DHT11 qua đường truyền 1-Wire Single-Bus, xuất dữ liệu trực quan lên màn hình tinh thể lỏng LCD 1602 (chế độ 4-bit bus) và kích hoạt dàn đèn LED cảnh báo khi nhiệt độ đo được vượt ngưỡng an toàn (30°C).

So với các giải pháp đo nhiệt độ sử dụng cảm biến tương tự truyền thống (như LM35), việc sử dụng cảm biến số DHT11 mang lại nhiều ưu điểm vượt trội: khả năng chống nhiễu cao trên đường truyền tín hiệu số, tích hợp đo đồng thời nhiệt độ và độ ẩm môi trường, kiểm tra mã kiểm tra lỗi (Checksum 8-bit) giúp xác thực tính toàn vẹn của gói dữ liệu thu thập. Đồng thời, đồ án cũng giải quyết triệt để các bài toán kỹ thuật thực tế trên Kit phần cứng như xử lý xung đột chân ngoại vi JTAG trên PORT C, cấu hình nguồn xung nhịp dao động nội 8.0 MHz bằng Fuse Bits và tối ưu hóa tài nguyên chân xuất nhập GPIO.

Báo cáo đồ án được kết cấu thành 4 chương chính như sau:

- **Chương 1: Giới thiệu tổng quan phần cứng và linh kiện**: Trình bày kiến trúc vi điều khiển ATmega16, Kit thực hành AVR chuẩn Bách Khoa, nguyên lý cảm

biến số DHT11, màn hình LCD 1602 và khối chỉ thị LED.

- **Chương 2: Thiết kế và triển khai hệ thống**: Trình bày thiết kế sơ đồ nguyên lý kết nối, lưu đồ thuật toán tổng thể, giải thuật giao tiếp 1-Wire, thư viện điều khiển LCD 1602 chế độ 4-bit, xử lý mặt nạ bit điều khiển cảnh báo và quy trình tự động hóa biên dịch, nạp firmware.
- **Chương 3: Kết quả thực nghiệm**: Trình bày kết quả thử nghiệm thực tế của hệ thống qua các kịch bản: khởi động, đo lường nhiệt độ bình thường, cảnh báo quá nhiệt và phản ứng an toàn khi mất tín hiệu cảm biến.
- **Chương 4: Kết luận và hướng phát triển**: Tổng kết các kết quả đạt được, chỉ ra những điểm còn hạn chế và đề xuất giải pháp phát triển mở rộng hệ thống lên nền tảng IoT.

Chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất tới thầy ThS. Vũ Hồng Vinh, giảng viên Trường Điện – Điện tử, Đại học Bách khoa Hà Nội, người đã tận tình định hướng, chỉ bảo và tạo mọi điều kiện thuận lợi nhất giúp chúng em hoàn thành tốt đồ án này. Chúng em cũng xin trân trọng cảm ơn các thầy cô giáo trong Trường Điện – Điện tử đã trang bị cho chúng em những nền tảng kiến thức quý báu trong suốt quá trình học tập.

Do thời gian thực hiện có hạn và kinh nghiệm thực tế của bản thân còn hạn chế, báo cáo chắc chắn không tránh khỏi những thiếu sót nhất định. Chúng em rất mong nhận được những lời nhận xét, góp ý quý báu của thầy cô và các bạn để đề tài được hoàn thiện hơn nữa.

Hà Nội, tháng 11 năm 2026

Sinh viên thực hiện

Mai Duy Hiếu

Hoàng Quốc Hưng

MỤC LỤC

LỜI NÓI ĐẦU **i**

DANH MỤC HÌNH VẼ **v**

DANH MỤC BẢNG **vi**

1 GIỚI THIỆU TỔNG QUAN PHẦN CỨNG VÀ LINH KIỆN **1**

1.1 Vi điều khiển ATmega16 1

1.1.1 Tổng quan kiến trúc và đặc tính kỹ thuật 1

1.1.2 Cấu trúc các cổng vào/ra (I/O Ports) và chức năng trong đồ án . . 2

1.1.3 Xung nhịp hệ thống và phân tích xung đột ngoại vi JTAG 3

1.2 Mạch Kit thực hành vi điều khiển họ AVR 4

1.3 Cảm biến số nhiệt độ và độ ẩm DHT11 6

1.3.1 Cấu tạo và nguyên lý hoạt động 6

1.3.2 Giao thức truyền thông 1-Wire Single-Bus 7

1.4 Màn hình hiển thị tinh thể lỏng LCD 1602 8

1.5 Khối cảnh báo và chỉ thị LED 10

2 THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG **11**

2.1 Thiết kế kiến trúc phần cứng và sơ đồ nguyên lý 11

2.1.1 Sơ đồ khối chức năng và kết nối liên khối hệ thống 11

2.1.2 Bảng kết nối dây chi tiết giữa các linh kiện 13

2.2 Xây dựng thuật toán và phần mềm điều khiển 14

2.2.1 Thuật toán điều khiển tổng quát chương trình 14

2.2.2 Thuật toán giao tiếp và giải mã cảm biến DHT11 15

2.2.3 Xây dựng thư viện điều khiển màn hình LCD 1602 16

2.2.4 Thuật toán so sánh ngưỡng cảnh báo và điều khiển LED 17

2.3 Quy trình biên dịch và nạp firmware 18
2.3.1 Quy trình tự động hóa với Makefile 18
2.3.2 Quy trình nạp chương trình bằng phần mềm ProgISP 19

3 KẾT QUẢ THỰC NGHIỆM 20

3.1 Kết quả thực nghiệm trên phần cứng 20
3.1.1 Giai đoạn khởi động và màn hình chào mừng 20
3.1.2 Trạng thái đo lường bình thường ($T < 30°\text{C}$) 20
3.1.3 Trạng thái cảnh báo vượt ngưỡng quá nhiệt ($T \geq 30°\text{C}$) 21
3.1.4 Kiểm tra phản hồi khi mất cảm biến và xử lý lỗi hệ thống 22
3.2 Đánh giá kết quả thực nghiệm 23

4 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 24

4.1 Kết luận 24
4.2 Hạn chế của đề tài 25
4.3 Hướng phát triển mở rộng 25

TÀI LIỆU THAM KHẢO 27

PHỤ LỤC: TOÀN BỘ MÃ NGUỒN CHƯƠNG TRÌNH 28

DANH MỤC HÌNH VẼ

1.1 Sơ đồ phân bố chân của vi điều khiển ATmega16 dạng đóng vỏ DIP-40 . 1

1.2 Sơ đồ bố trí linh kiện trên Kit phát triển vi điều khiển AVR 5

2.1 Sơ đồ kết nối liên khối chi tiết của hệ thống đo nhiệt độ và cảnh báo . . . 11

2.2 Sơ đồ nguyên lý mạch Kit vi điều khiển ATmega16 12

2.3 Mô hình thiết kế 3D của bo mạch Kit trên phần mềm Altium Designer . . 12

2.4 Giao diện phần mềm ProgISP khi nạp firmware và cấu hình Fuse Bits 0xD9E4 cho ATmega16 19

3.1 Hình ảnh thực tế hệ thống vận hành trong các kịch bản thử nghiệm 21

DANH MỤC BẢNG

1.1 Thông số kỹ thuật chính của vi điều khiển ATmega16 2

1.2 Đặc tính kỹ thuật của cảm biến số DHT11 6

1.3 Bảng ánh xạ kết nối chân giữa giắc J7 trên Kit, màn hình LCD 1602 và ATmega16 9

2.1 Bảng tổng hợp kết nối tín hiệu phần cứng của hệ thống 13

3.1 Bảng tổng hợp các kịch bản kiểm tra và kết quả đánh giá hệ thống . . . 23

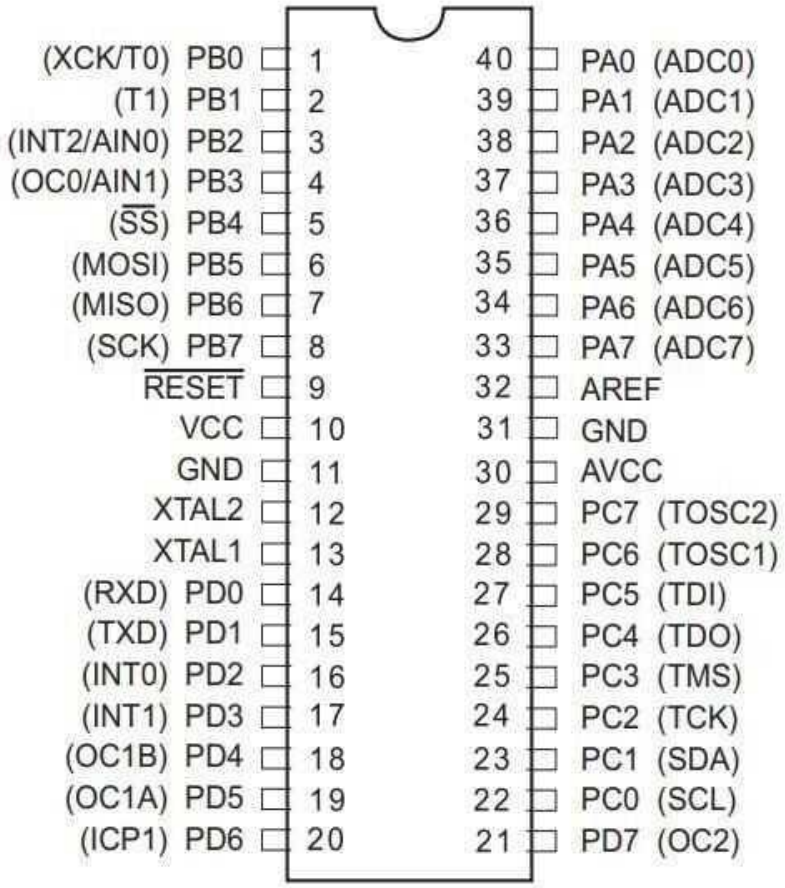
CHƯƠNG 1

GIỚI THIỆU TỔNG QUAN PHẦN CỨNG VÀ LINH KIỆN

1.1 Vi điều khiển ATmega16

1.1.1 Tổng quan kiến trúc và đặc tính kỹ thuật

ATmega16 là một trong những dòng vi điều khiển 8-bit họ AVR phổ biến nhất của hãng Atmel (nay thuộc Microchip Technology), được thiết kế dựa trên kiến trúc RISC (Reduced Instruction Set Computer) tiên tiến với tập lệnh rút gọn gồm 131 lệnh đa số thực thi trong một chu kỳ xung nhịp đơn lẻ [1]. Nhờ thông lượng xử lý đạt tới 1 MIPS trên mỗi MHz tần số hoạt động, vi điều khiển mang lại hiệu suất tính toán tối ưu song song với khả năng tiết kiệm năng lượng vượt trội.



Hình 1.1: Sơ đồ phân bố chân của vi điều khiển ATmega16 dạng đóng vỏ DIP-40

Các thông số kỹ thuật then chốt của vi điều khiển ATmega16 được tổng hợp trong Bảng 1.1:

Bảng 1.1: Thông số kỹ thuật chính của vi điều khiển ATmega16

Thông số	Giá trị định mức / Đặc tính kỹ thuật
Kiến trúc CPU	8-bit AVR RISC với 32 thanh ghi làm việc đa năng 8-bit
Bộ nhớ Flash chương trình	16 Kbytes (In-System Reprogrammable Flash, độ bền 10.000 lần ghi/xóa)
Bộ nhớ SRAM nội	1 Kbyte (1024 Bytes)
Bộ nhớ EEPROM nội	512 Bytes (Lưu trữ dữ liệu không mất khi mất điện, độ bền 100.000 lần)
Tần số xung nhịp tối đa	0 – 16 MHz (Điện áp 4.5V – 5.5V)
Số chân I/O khả trình	32 chân chia làm 4 cổng 8-bit (PORT A, PORT B, PORT C, PORT D)
Bộ định thời / Bộ đếm (Timer)	2 bộ Timer/Counter 8-bit và 1 bộ Timer/Counter 16-bit đa chức năng
Các kênh PWM	4 kênh điều chế độ rộng xung phần cứng (Pulse Width Modulation)
Bộ chuyển đổi ADC	8 kênh ADC 10-bit tích hợp sẵn trên PORT A
Giao tiếp truyền thông	USART lập trình được, SPI (Serial Peripheral Interface), I2C/TWI
Giao diện gỡ lỗi phần cứng	Chuẩn JTAG (IEEE std. 1149.1) hỗ trợ On-Chip Debug
Điện áp hoạt động	4.5V – 5.5V đối với ATmega16 chuẩn (2.7V – 5.5V đối với ATmega16L)

1.1.2 Cấu trúc các cổng vào/ra (I/O Ports) và chức năng trong đồ án

Vi điều khiển ATmega16 trang bị 32 chân vào ra hai chiều, được phân bổ đều cho 4 cổng ngoại vi ký hiệu từ A đến D. Mỗi cổng được kiểm soát độc lập bởi 3 thanh ghi điều khiển địa chỉ bộ nhớ:

- Thanh ghi DDRx (hướng dữ liệu): Thiết lập cấu hình hướng cho từng chân (Bit = 0: Chân là đầu vào Input; Bit = 1: Chân là đầu ra Output).
- Thanh ghi PORTx (dữ liệu xuất): Khi chân được thiết lập là Output, việc ghi 0

hoặc 1 vào bit tương ứng sẽ xuất điện áp mức thấp (0V) hoặc mức cao (5V). Khi chân là Input, ghi bit 1 sẽ kích hoạt điện trở kéo lên nội, ghi bit 0 sẽ đưa chân về trạng thái trở kháng cao.

- Thanh ghi PINx (đọc dữ liệu vào): Đọc trạng thái logic điện áp thực tế tại các chân cổng.

Trong phạm vi đề tài, các chân I/O được phân công đảm nhiệm các chức năng cụ thể như sau:

- Chân PA1 (cổng A): Đường truyền giao tiếp 1-Wire hai chiều với cảm biến số DHT11. Chân này liên tục chuyển đổi trạng thái giữa Output (khi MCU phát xung kích) và Input (khi MCU đọc chuỗi xung phản hồi từ cảm biến).
- Các chân PC4 – PC7 (cổng C): Bus truyền dữ liệu 4-bit (tương ứng các chân D4, D5, D6, D7) tới màn hình LCD 1602.
- Các chân PD5, PD6, PD7 (cổng D): Các đường tín hiệu điều khiển màn hình LCD 1602, lần lượt ứng với các tín hiệu RW, RS và EN.
- Các chân PD0 – PD4 (cổng D): Điều khiển dàn 5 đèn LED cảnh báo ngưỡng theo chế độ tích cực mức thấp.

1.1.3 Xung nhịp hệ thống và phân tích xung đột ngoại vi JTAG

Cấu hình nguồn xung nhịp 8.0 MHz bằng Fuse Bits

ATmega16 tích hợp sẵn mạch dao động RC nội (Calibrated Internal RC Oscillator) có khả năng tạo ra các tần số xung nhịp cố định gồm 1.0 MHz, 2.0 MHz, 4.0 MHz và 8.0 MHz mà không đòi hỏi phải gắn thêm thạch anh dao động ngoài hay tụ lọc phụ trợ.

Trên Kit thực hành vi điều khiển của Trường Điện – Điện tử (ĐHBK Hà Nội), vị trí thạch anh ngoài `XT1` để trống và không hàn tụ hồi tiếp. Do đó, việc cấu hình vi điều khiển chạy bằng dao động RC nội tần số 8.0 MHz là bắt buộc để hệ thống đạt tốc độ xử lý nhanh, đồng thời triệt tiêu rủi ro chip bị khóa xung nhịp. Cấu hình này được xác lập thông qua thanh ghi cầu chì (Fuse Bits) với giá trị:

- Low Fuse Byte = `0xE4`: Thiết lập các bit `CKSEL3..0 = 0100` chọn mạch dao động RC nội 8.0 MHz, thời gian khởi động ổn định nguồn xung là 64 ms (`SUT1..0 = 10`).

Phân tích xung đột phần cứng JTAG trên PORT C

Mặc định khi xuất xưởng từ nhà sản xuất, bit cầu chì `JTAGEN` trên High Fuse Byte luôn được kích hoạt sẵn (`JTAGEN = 0`). Khi giao tiếp JTAG hoạt động, phần cứng vi điều khiển sẽ chiếm dụng cố định 4 chân trên cổng C:

- Chân PC2 = TCK (Test Clock)
- Chân PC3 = TMS (Test Mode Select)
- Chân PC4 = TDO (Test Data Output)
- Chân PC5 = TDI (Test Data Input)

Do bus dữ liệu 4-bit của màn hình LCD 1602 được nối vào các chân PC4 – PC7, việc tính năng JTAG bị bật sẽ khiến hai chân PC4 và PC5 bị treo cưỡng bức ở mức logic của khối gỡ lỗi, làm vô hiệu hóa khả năng xuất dữ liệu GPIO của CPU. Hệ quả là màn hình LCD không thể nhận được các byte lệnh khởi tạo, dẫn tới hiện tượng lỗi kinh điển: hàng 1 màn hình nổi toàn bộ khối đen và hàng 2 trắng tinh.

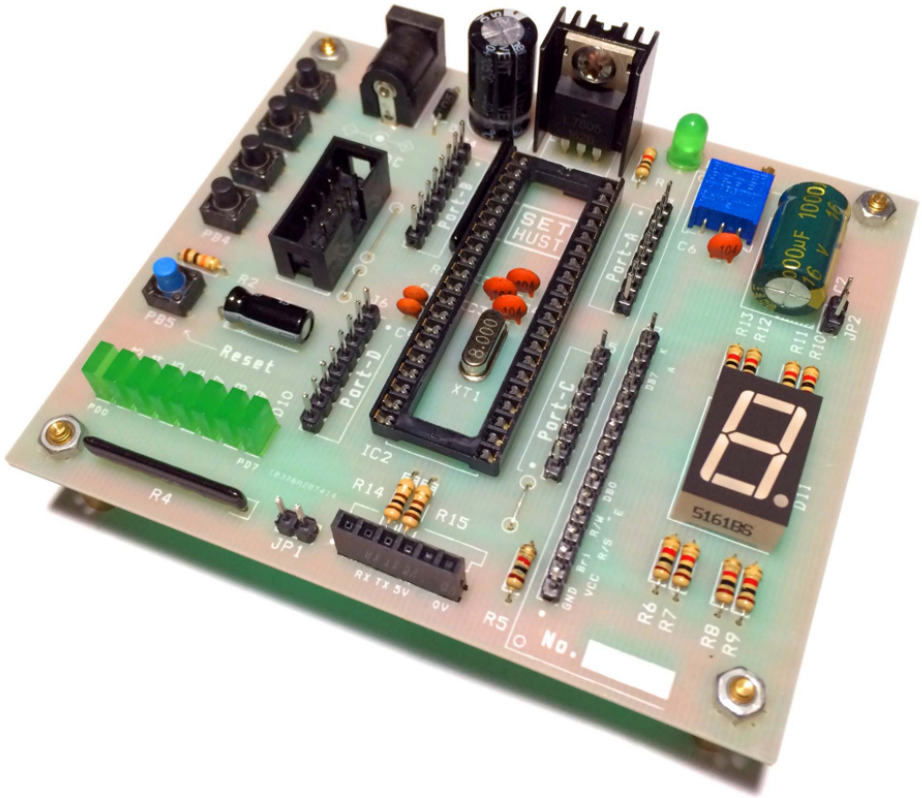
Để giải phóng hoàn toàn PORT C cho giao tiếp LCD, đề tài áp dụng giải pháp kép:

1. Tắt JTAG ở mức phần cứng: Cấu hình High Fuse Byte = `0xD9`. Tại giá trị này, bit 6 (`JTAGEN`) mang giá trị 1 (vô hiệu hóa JTAG), bit 5 (`SPIEN`) mang giá trị 0 (duy trì cổng nạp SPI để nạp code qua USBasp).
2. Tắt JTAG ở mức phần mềm: Ghi bit `JTD` trên thanh ghi `MCUCSR` hai lần liên tiếp trong vòng 4 chu kỳ máy ngay ở đầu hàm `main()`:

```
MCUCSR = (1 << JTD);
MCUCSR = (1 << JTD);
```

1.2 Mạch Kit thực hành vi điều khiển họ AVR

Mạch Kit thực hành vi điều khiển AVR sử dụng trong đề tài là module chuẩn được nghiên cứu và thiết kế phục vụ công tác giảng dạy, thí nghiệm tại Trường Điện – Điện tử, Đại học Bách khoa Hà Nội (Hình 1.2).



Hình 1.2: Sơ đồ bố trí linh kiện trên Kit phát triển vi điều khiển AVR

Kit phát triển tích hợp đầy đủ các khối chức năng ngoại vi cơ bản:

1. Đế cắm vi điều khiển DIP-40 (ZIF Socket): Giúp việc tháo lắp IC vi điều khiển dễ dàng, an toàn, tránh cong gãy chân cắm.
2. Khối nguồn nuôi và ổn áp 5V: Tích hợp IC ổn áp tuyến tính LM7805 hạ áp từ nguồn Adapter ngoài DC 9V–12V, đồng thời hỗ trợ nhận nguồn 5V trực tiếp từ cổng USB qua mạch nạp USBasp, kết hợp hệ thống tụ lọc nguồn và đèn LED chỉ thị nguồn VCC.
3. Giao diện nạp chương trình chuẩn ISP 10 chân (IDC-10): Kết nối trực tiếp với mạch nạp USBasp qua các đường tín hiệu MOSI (PB5), MISO (PB6), SCK (PB7), RESET và GND.
4. Hàng rào cắm mở rộng J7 cho LCD 1602: Chuẩn cắm hàng đơn 16 chân tương thích với các màn hình chuẩn HD44780, tích hợp sẵn biến trở vi chỉnh VR1 dùng để điều chỉnh độ tương phản.
5. Dàn LED đơn và phím bấm chức năng: Hệ thống 8 LED đơn nối về PORT D và ma trận nút bấm điều khiển nối về PORT B phục vụ các bài toán thực hành giao

tiếp vào ra cơ bản.

1.3 Cảm biến số nhiệt độ và độ ẩm DHT11

1.3.1 Cấu tạo và nguyên lý hoạt động

DHT11 là cảm biến số đo nhiệt độ và độ ẩm môi trường tích hợp vi xử lý 8-bit hiệu năng cao bên trong để số hóa và hiệu chuẩn dữ liệu trước khi truyền ra ngoài [2]. Cấu tạo cảm biến bao gồm hai thành phần cảm nhận vật lý:

- Cảm biến độ ẩm kiểu điện trở (Resistive Humidity Sensor): Sử dụng một phần tử màng polymer nhạy ẩm có các điện cực xen kẽ. Khi độ ẩm môi trường biến thiên, các phân tử hơi nước được hấp thụ vào bề mặt làm phân ly các ion dẫn, dẫn đến biến đổi điện trở của màng. Vi điều khiển 8-bit tích hợp bên trong DHT11 sẽ đo đạc sự thay đổi điện trở này và tính toán quy đổi thành giá trị độ ẩm tương đối (%RH).
- Cảm biến nhiệt độ NTC: Là một điện trở nhiệt bán dẫn có hệ số nhiệt điện trở âm. Khi nhiệt độ môi trường tăng lên, điện trở của linh kiện giảm xuống theo một hàm phi tuyến được định trước. Vi điều khiển chuyển đổi giá trị điện trở này thành nhiệt độ thực tế theo thang đo độ Celsius (°C).

Bảng 1.2: Đặc tính kỹ thuật của cảm biến số DHT11

Thông số kỹ thuật	Giá trị danh định
Điện áp hoạt động	3.3V – 5.5V DC
Dải đo nhiệt độ	0°C – 50°C
Độ chính xác đo nhiệt độ	±2.0°C
Độ phân giải nhiệt độ	1.0°C (hoặc 0.1°C ở một số bản nâng cấp)
Dải đo độ ẩm	20% – 90% RH
Độ chính xác đo độ ẩm	±5.0% RH
Độ phân giải độ ẩm	1% RH
Chu kỳ lấy mẫu tối thiểu	$\geq$ 1.0 giây (Khuyến nghị 2.0 giây theo tài liệu của Aosong để đảm bảo ổn định)
Dòng điện tiêu thụ	0.5 mA – 2.5 mA khi đo; 100 μA – 150 μA khi chờ
Chuẩn giao tiếp	1-Wire Single-Bus (Đường truyền dữ liệu đơn tuyến số)

1.3.2 Giao thức truyền thông 1-Wire Single-Bus

Cảm biến DHT11 giao tiếp với vi điều khiển chỉ thông qua một chân dữ liệu DATA duy nhất. Đường tín hiệu này đòi hỏi phải có một điện trở kéo lên nguồn 5V (giá trị từ 4.7 kΩ đến 10 kΩ) để duy trì mức cao khi đường truyền ở trạng thái rảnh rỗi.

Toàn bộ quy trình trao đổi dữ liệu một lần đọc (gồm 40 bit) tuân theo trình tự định thì thời gian nghiêm ngặt được mô tả dưới đây:

1. Tín hiệu khởi động

Vi điều khiển phát tín hiệu khởi động:

- MCU cấu hình chân PA1 thành Output và kéo đường bus xuống mức LOW trong thời gian tối thiểu 18 ms (trong chương trình chọn trễ an toàn là `_delay_ms(20)`).
- MCU sau đó giải phóng quyền điều khiển bus bằng cách chuyển chân PA1 sang chế độ Input và kích hoạt điện trở kéo lên nội, tạo khoảng trễ 30 μs để đường tín hiệu được kéo lên mức HIGH an toàn trước khi cảm biến bắt đầu phản hồi. Trình tự này bảo đảm tránh triệt để hiện tượng xung đột mức logic (Bus Contention) trên đường truyền chung.

2. Tín hiệu phản hồi từ cảm biến

Khi nhận diện được xung khởi động hợp lệ, DHT11 sẽ kéo đường bus xuống mức LOW trong khoảng 80 μs, tiếp theo kéo lên mức HIGH trong 80 μs để thông báo đã sẵn sàng truyền gói dữ liệu.

3. Thu nhận và giải mã 40 bit dữ liệu

Sau khi gửi tín hiệu phản hồi, DHT11 bắt đầu tuần tự phát ra 40 bit nhị phân (chia đều thành 5 byte). Mỗi bit dữ liệu luôn được bắt đầu bằng một xung mức LOW kéo dài cố định khoảng 50 μs. Giá trị của bit nhị phân được mã hóa dựa trên độ rộng thời gian tồn tại của xung mức HIGH tiếp theo đó:

- Bit ’0’: Xung mức HIGH tồn tại trong khoảng từ 26 μs đến 28 μs.
- Bit ’1’: Xung mức HIGH tồn tại kéo dài trong khoảng 70 μs.

Dựa vào sự chênh lệch thời gian rõ rệt này, vi điều khiển áp dụng kỹ thuật lấy mẫu:

Sau khi phát hiện kết thúc xung LOW 50 μs, MCU thực hiện trễ phần mềm một khoảng thời gian trung gian bằng 38 μs rồi đọc trạng thái logic chân PA1:

- Nếu chân PA1 vẫn còn ở mức HIGH $\rightarrow$ Độ rộng xung lớn hơn 38 μs, đây là bit '1'.
- Nếu chân PA1 đã rơi xuống mức LOW $\rightarrow$ Xung đã kết thúc trước thời điểm 38 μs, đây là bit '0'.

4. Cấu trúc gói tin và cơ chế kiểm tra lỗi

Gói dữ liệu hoàn chỉnh gồm 5 byte (40 bit) truyền đi theo thứ tự:

$$\begin{aligned}\text{Gói dữ liệu} = \{&\text{Byte 0: } RH_{int}, \quad \text{Byte 1: } RH_{dec}, \\ &\text{Byte 2: } T_{int}, \quad \text{Byte 3: } T_{dec}, \quad \text{Byte 4: } Checksum\}\end{aligned} \tag{1.1}$$

Byte cuối cùng là byte kiểm tra lỗi Checksum được tạo bởi tổng đại số của 4 byte dữ liệu trước đó (lấy 8 bit thấp):

$$Checksum_{valid} = (RH_{int} + RH_{dec} + T_{int} + T_{dec}) \text{ \& 0xFF} \tag{1.2}$$

Nếu giá trị tính toán trùng khớp với Byte 4 nhận được từ cảm biến, gói dữ liệu được xác nhận là hợp lệ. Thuật toán Checksum 8-bit giúp phát hiện hiệu quả các trường hợp méo dạng xung hoặc sai lệch bit đơn lẻ trên đường truyền; tuy nhiên, phương pháp tổng kiểm tra cộng 8-bit không hỗ trợ tự sửa sai (Error Correction) và không thể loại trừ hoàn toàn các lỗi đảo bit bù trừ đa điểm.

1.4 Màn hình hiển thị tinh thể lỏng LCD 1602

Màn hình LCD 1602 là mô-đun hiển thị ký tự (Character LCD) gồm 2 hàng, mỗi hàng có khả năng hiển thị tối đa 16 ký tự ma trận điểm 5×8 pixels, sử dụng vi mạch điều khiển tiêu chuẩn công nghiệp Hitachi HD44780 [3].

Bảng 1.3: Bảng ánh xạ kết nối chân giữa giắc J7 trên Kit, màn hình LCD 1602 và ATmega16

Chân J7	Chân LCD	Tên chân	**Chân MCU**	Chức năng kỹ thuật
1	1	V_{SS}	**GND**	Cực âm nguồn nuôi (0V)
2	2	V_{DD}	**VCC**	Cực dương nguồn nuôi (+5V)
3	3	V_0	**VR1**	Điện áp điều chỉnh độ tương phản hiển thị màn hình
4	4	RS	**PD6**	Chọn thanh ghi (0: Thanh ghi Lệnh; 1: Thanh ghi Dữ liệu)
5	5	RW	**PD5**	Chọn chiều Đọc/Ghi (0: Ghi dữ liệu vào LCD; 1: Đọc từ LCD)
6	6	E	**PD7**	Chốt xung kích hoạt (Enable Strobe, tích cực sườn âm)
7 – 10	7 – 10	D0 – D3	**PC0 – PC3**	4 bit dữ liệu thấp (Không dùng ở chế độ 4-bit bus)
11	11	D4	**PC4**	Bit dữ liệu số 4 (Data line 4)
12	12	D5	**PC5**	Bit dữ liệu số 5 (Data line 5)
13	13	D6	**PC6**	Bit dữ liệu số 6 (Data line 6)
14	14	D7	**PC7**	Bit dữ liệu số 7 (Data line 7)
15	15	A (LED+)	**VCC**	Cực Anot đèn nền LED nền màn hình (Backlight)
16	16	K (LED-)	**GND**	Cực Katot đèn nền LED nền màn hình (Backlight)

Để tối ưu hóa số lượng chân vi điều khiển, đồ án áp dụng chế độ truyền thông 4-bit bus:

- Vi điều khiển chỉ cần dùng 4 chân GPIO (PC4 – PC7) nối tới các chân dữ liệu D4 – D7 của LCD, tiết kiệm được 4 chân so với chế độ 8-bit truyền thống.
- Mỗi byte dữ liệu hoặc mã lệnh 8-bit sẽ được tách làm hai nửa byte (Nibble): 4 bit cao được truyền trước kèm theo một xung chốt Enable trên chân PD7, tiếp theo 4 bit thấp được dịch sang và truyền kèm theo một xung chốt Enable thứ hai.

1.5 Khối cảnh báo và chỉ thị LED

Dàn 5 đèn LED cảnh báo trên Kit thí nghiệm được nối trực tiếp vào các chân từ PD0 đến PD4 của cổng D. Mạch được thiết kế theo cấu hình tích cực mức thấp (Active Low):

- Cực Anot của các đèn LED được cấp nguồn dương VCC thông qua điện trở hạn dòng 330 Ω.
- Cực Katot của các đèn LED nối về các chân vi điều khiển PD0 – PD4.
- Khi vi điều khiển xuất mức 0 (mức thấp): Dòng điện chạy từ VCC qua LED xuống chân vi điều khiển khép mạch, LED bật sáng (báo động vượt ngưỡng).
- Khi vi điều khiển xuất mức 1 (mức cao): Không có chênh lệch điện áp qua diode, LED tắt (trạng thái nhiệt độ an toàn).

CHƯƠNG 2

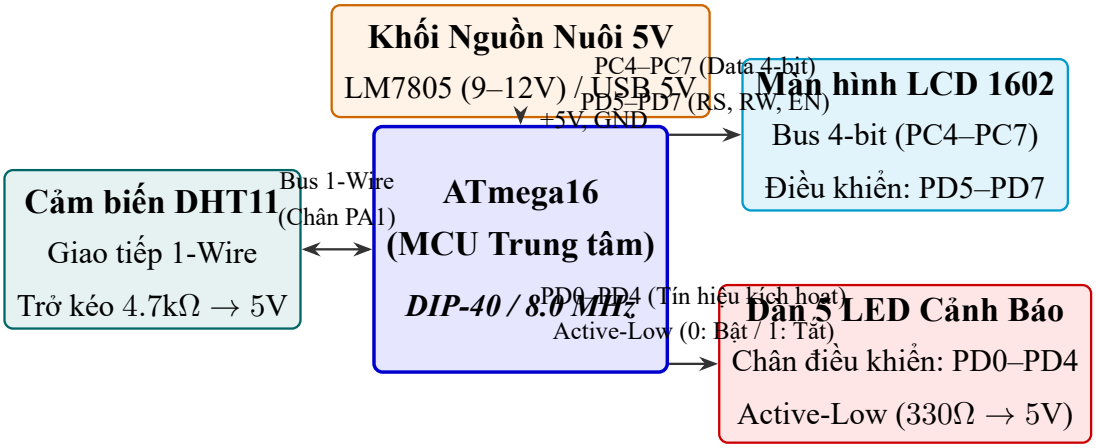
THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

2.1 Thiết kế kiến trúc phần cứng và sơ đồ nguyên lý

2.1.1 Sơ đồ khối chức năng và kết nối liên khối hệ thống

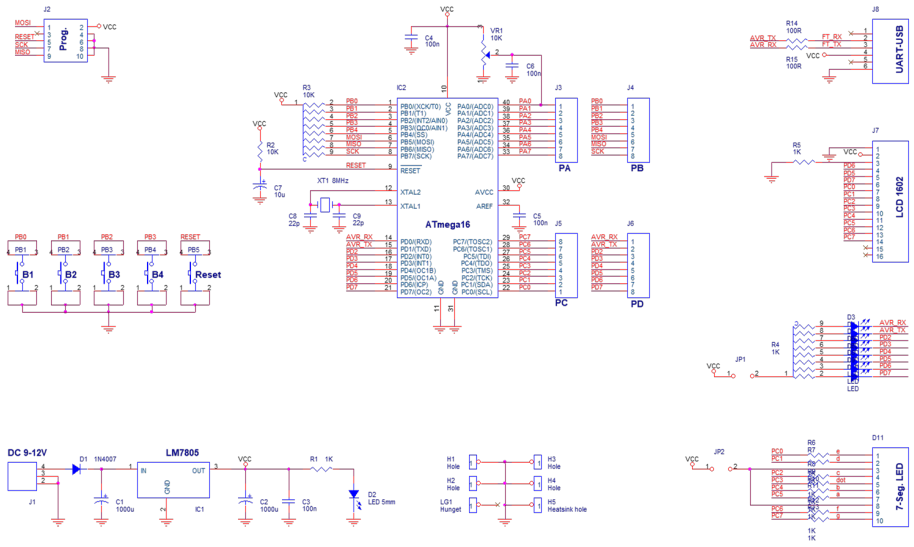
Hệ thống đo nhiệt độ và cảnh báo ngưỡng được xây dựng trên cấu trúc liên kết gồm 5 khối chức năng chính:

1. **Khối nguồn nuôi**: Cung cấp điện áp ổn định +5V DC qua hai đường: nguồn ngoài 9–12V qua IC ổn áp LM7805 trên Kit, hoặc cấp 5V trực tiếp từ cổng nạp USBasp/USB máy tính.
2. **Khối MCU trung tâm (ATmega16)**: Thực thi chương trình nhúng, phát xung kích hoạt 1-Wire, nhận gói tin và kiểm tra Checksum, so sánh ngưỡng nhiệt độ 30°C, điều khiển LCD 1602 và dàn LED cảnh báo.
3. **Khối cảm biến số DHT11**: Thu thập nhiệt độ, độ ẩm môi trường và truyền dữ liệu nối tiếp về chân PA1 (tích hợp điện trở kéo lên 4.7kΩ lên nguồn 5V).
4. **Khối hiển thị LCD 1602**: Kết nối ở chế độ 4-bit bus qua các chân PC4–PC7 (dữ liệu) và PD5–PD7 (điều khiển), hiển thị thời gian thực thông số đo và cảnh báo.
5. **Khối cảnh báo**: Dàn 5 LED đơn nối với các chân PD0–PD4, tích cực mức thấp (Active Low) qua trở hạn dòng 330Ω.

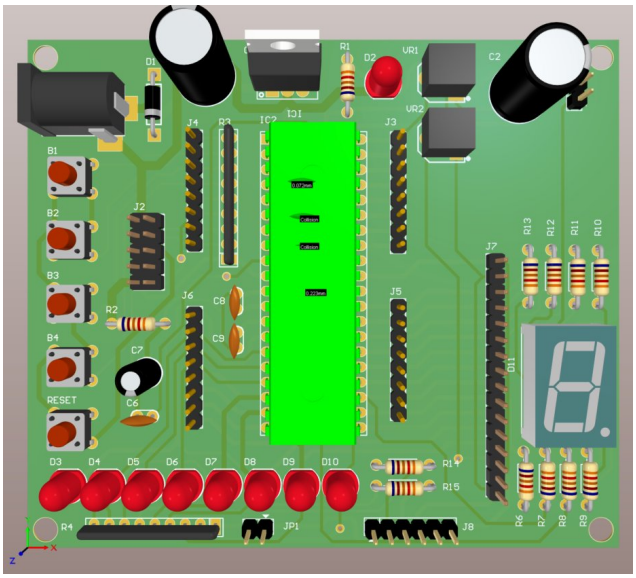


Hình 2.1: Sơ đồ kết nối liên khối chi tiết của hệ thống đo nhiệt độ và cảnh báo

Sơ đồ nguyên lý bo mạch Kit phát triển ATmega16 và mô hình không gian 3D hoàn thiện trên Altium Designer được thể hiện lần lượt trên Hình 2.2 và Hình 2.3:



Hình 2.2: Sơ đồ nguyên lý mạch Kit vi điều khiển ATmega16



Hình 2.3: Mô hình thiết kế 3D của bo mạch Kit trên phần mềm Altium Designer

2.1.2 Bảng kết nối dây chi tiết giữa các linh kiện

Bảng 2.1: Bảng tổng hợp kết nối tín hiệu phần cứng của hệ thống

Khối chức năng	**Chân linh kiện**	Chân MCU	**Tên tín hiệu MCU**	Chức năng kỹ thuật
Cảm biến DHT11	**DATA (Chân 2)**	Chân 39	**PA1 / ADC1**	Giao tiếp 1-Wire hai chiều
	VCC (Chân 1)	Chân 10, 30	**VCC, AVCC**	Nguồn nuôi +5V DC
	GND (Chân 4)	Chân 11, 31	**GND**	Nối đất chung (Mass hệ thống)
Màn hình LCD	**RS (Chân 4)**	Chân 20	**PD6 / ICP1**	Chọn thanh ghi LCD (0: Lệnh, 1: Data)
	RW (Chân 5)	Chân 19	**PD5 / OC1A**	Chọn chiều đọc/ghi (Kéo 0: Ghi)
	EN (Chân 6)	Chân 21	**PD7 / OC2**	Xung kích hoạt chốt dữ liệu
	D4 (Chân 11)	Chân 26	**PC4 / TDO**	Đường truyền dữ liệu bit 4
	D5 (Chân 12)	Chân 27	**PC5 / TDI**	Đường truyền dữ liệu bit 5
	D6 (Chân 13)	Chân 28	**PC6 / TOSC1**	Đường truyền dữ liệu bit 6
	D7 (Chân 14)	Chân 29	**PC7 / TOSC2**	Đường truyền dữ liệu bit 7
Dàn LED cảnh báo	**Katot LED 1**	Chân 14	**PD0 / RXD**	Báo động ngưỡng (Active Low)
	Katot LED 2	Chân 15	**PD1 / TXD**	Báo động ngưỡng (Active Low)
	Katot LED 3	Chân 16	**PD2 / INT0**	Báo động ngưỡng (Active Low)
	Katot LED 4	Chân 17	**PD3 / INT1**	Báo động ngưỡng (Active Low)
	Katot LED 5	Chân 18	**PD4 / OC1B**	Báo động ngưỡng (Active Low)

2.2 Xây dựng thuật toán và phần mềm điều khiển

2.2.1 Thuật toán điều khiển tổng quát chương trình

Quy trình vận hành của hệ thống được thực thi tuần tự và khép kín trong vòng lặp vô hạn theo các bước logic:

1. Khởi tạo hệ thống:
    - Thực thi hai lệnh liên tiếp tắt tính năng JTAG trên thanh ghi `MCUCSR` để giải phóng các chân PC4 và PC5 cho cổng LCD.
    - Khởi tạo cấu hình màn hình LCD 1602 ở chế độ 4-bit bus (`LCD4_INIT`).
    - Cấu hình 5 chân thấp của cổng D (`PD0 -- PD4`) làm cổng ra Output (`DDRD |= 0x1F`), đồng thời xuất mức logic 1 (`PORTD |= 0x1F`) để tắt toàn bộ dàn LED cảnh báo ban đầu.
    - Xuất thông điệp chào mừng lên LCD: Hàng 1 in chuỗi `"DA2: DO NHIET DO"`, Hàng 2 in chuỗi `"DHT11 SENSOR"`, duy trì trong 1.5 giây sau đó xóa màn hình.
2. Vòng lặp đo lường và xử lý chính:
    - Gọi hàm `DHT11_Read` để bắt đầu quá trình giao tiếp 1-Wire và giải mã dữ liệu cảm biến.
    - **Nhánh 1 – Đọc thành công (`status == 0`)**:
        - Dòng 1 LCD: Định dạng và hiển thị nhiệt độ `"Temp: XX.Y °C "` (đúng 16 ký tự).
        - Dòng 2 LCD: So sánh phần nguyên của nhiệt độ (`temp_int`) với ngưỡng $30°$C:
            - Nếu $temp_int \geq 30°$C: In chuỗi cảnh báo `"HIGH Th: 30°C *!"` (16 ký tự) và kéo các chân PD0–PD4 xuống mức 0 (`PORTD &= 0xE0`) để bật sáng cả 5 LED.
            - Nếu $temp_int < 30°$C: In chuỗi `"HIGH Th: 30°C "` (16 ký tự, xóa ký hiệu báo động cũ) và kéo các chân PD0–PD4 lên mức 1 (`PORTD |= 0x1F`) để tắt dàn LED.
    - **Nhánh 2 – Lỗi sai mã Checksum (`status == -6`)**: Dòng 1 in `"DHT11: Sum Error"`, dòng 2 in `"Checksum Invalid"`, đưa dàn 5 LED về trạng thái tắt xác định (`PORTD |= 0x1F`).

- **Nhánh 3 – Lỗi mất kết nối / Timeout (`status` từ -1 đến -5)**: Dòng 1 in `"DHT11: No Signal"`, dòng 2 in `"Check Pin PA1 "`, đưa dàn 5 LED về trạng thái tắt xác định (`PORTD |= 0x1F`).
- Tạo trễ phần mềm 2.0 giây (`_delay_ms(2000)`) trước khi bắt đầu chu kỳ đọc tiếp theo theo khuyến cáo từ tài liệu của Aosong.

2.2.2 Thuật toán giao tiếp và giải mã cảm biến DHT11

Hàm `DHT11_Read(uint8_t *temp_int, uint8_t *temp_dec, uint8_t *humi_int)` được xây dựng trực tiếp trên các thao tác thanh ghi bare-metal nhằm đảm bảo độ chính xác tối đa về mặt thời gian (Timing), đồng thời tích hợp cơ chế bảo vệ Timeout chống treo vi điều khiển.

Cơ chế chống treo CPU (Timeout Protection)

Trong các hàm giao tiếp nối tiếp không đồng bộ, nếu chân tín hiệu bị hở mạch hoặc cảm biến hỏng, các vòng lặp đợi sườn xung `while(DHT_PIN_R & (1 << DHT_BIT))` sẽ trở thành vòng lặp vô tận (Infinite Loop), làm vi điều khiển bị treo hoàn toàn (Freeze). Để giải quyết triệt để lỗi này, một biến đếm giảm dần `timeout = 10000` được gài vào tất cả các vòng lặp chờ. Nếu sau 10.000 chu kỳ lặp mà trạng thái chân vẫn không đổi, hàm lập tức thoát ra và trả về mã lỗi âm tương ứng (từ -1 đến -5), giúp hệ thống tiếp tục vận hành an toàn và đưa ra cảnh báo lỗi lên màn hình.

Thuật toán đo độ rộng xung nhị phân

Sau khi vượt qua xung Start và tín hiệu phản hồi từ DHT11, vòng lặp lồng nhau quét qua 5 byte (40 bit). Mỗi bit nhị phân được giải mã theo các bước:

1. Đợi kết thúc xung mức LOW 50 μs của bit đang truyền.
2. Đưa CPU vào trạng thái trễ 38 μs (`_delay_us(38)`).
3. Đọc thanh ghi `PINA`: nếu bit PA1 đang ở mức 1, ta ghi nhận đây là bit ’1’ bằng phép dịch bit:

```
data[i] |= (1 << j);
```

Tiếp tục chờ xung mức HIGH kết thúc trước khi đón nhận bit tiếp theo.

Kiểm tra tính toàn vẹn dữ liệu (Checksum) và phân loại lỗi

Cuối hàm `DHT11_Read`, tổng của 4 byte đầu được so sánh với byte thứ 5 (byte kiểm tra lỗi Checksum):

```
if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4]) {
    return -6; // Lỗi sai Checksum
}
```

Nếu kiểm tra thành công, các con trỏ nhận giá trị tương ứng và hàm trả về mã 0 (Thành công).

Thuật toán trong hàm chính `main()` thực hiện phân loại lỗi một cách tường minh:

- Khi xảy ra lỗi Timeout (mã lỗi từ -1 đến -5, tương ứng với sự cố đứt dây hoặc mất tín hiệu từ cảm biến): Màn hình LCD hiển thị thông báo `"DHT11: No Signal"` và hướng dẫn `"Check Pin PA1"`.
- Khi xảy ra lỗi sai Checksum (mã lỗi -6, tương ứng với trường hợp gói dữ liệu bị méo dạng do nhiễu môi trường): Màn hình LCD hiển thị `"DHT11: Sum Error"` và `"Checksum Invalid"`.
- Trạng thái xử lý lỗi xác định (Deterministic Error Handling): Trong mọi trường hợp xảy ra lỗi cảm biến, vi điều khiển chủ động đưa dàn 5 LED cảnh báo về mức tắt hoàn toàn (`PORTD |= 0x1F`) nhằm loại bỏ nguy cơ lưu giữ báo động giả từ dữ liệu cũ lỗi thời, đồng thời tập trung hiển thị chẩn đoán lỗi trực quan trên LCD. Tuy nhiên, giải pháp này có sự đánh đổi kỹ thuật (Trade-off): nếu trước khi cảm biến gặp sự cố mà nhiệt độ thực tế đang ở mức cao (trên $30°C$), việc tắt LED sẽ khiến hệ thống không còn cảnh báo quá nhiệt bằng ánh sáng. Đây là một điểm hạn chế cần được cải tiến bằng chế độ nhấp nháy đèn báo lỗi riêng biệt.

2.2.3 Xây dựng thư viện điều khiển màn hình LCD 1602

Thư viện `hunget_lcd.h` được thiết kế tối ưu hóa cho phần cứng Kit vi điều khiển của Trường Điện – Điện tử.

Hàm tạo xung chốt dữ liệu (`LCD_Enable`)

Theo chuẩn Hitachi HD44780, dữ liệu tại bus 4-bit chỉ được bộ điều khiển LCD ghi nhận tại thời điểm sườn âm (Falling edge) của chân Enable (chân PD7). Do vi điều

khiển chạy ở tần số 8.0 MHz (mỗi chu kỳ lệnh mất $0.125\ \mu$s), hàm tạo xung cần đảm bảo thời gian thiết lập (Setup time) và thời gian duy trì (Hold time):

```
static inline void LCD_Enable(void) {
    LCD_CTRL_PORT |= (1 << LCD_EN);
    _delay_us(5);
    LCD_CTRL_PORT &= ~(1 << LCD_EN);
    _delay_us(100);
}
```

Hàm truyền 4 bit dữ liệu (`LCD_Send4Bit`)

Do cổng PORT C chỉ dùng 4 chân cao (PC4 – PC7) cho LCD, việc ghi dữ liệu ra LCD tuyệt đối không được làm thay đổi trạng thái của 4 chân thấp (PC0 – PC3). Do đó, phép toán mặt nạ bit (Bitwise AND/OR) được áp dụng:

```
static inline void LCD_Send4Bit(unsigned char Data) {
    unsigned char current_port_c = LCD_DATA_PORT & 0x0F;
    LCD_DATA_PORT = current_port_c | (Data & 0xF0);
    LCD_Enable();
}
```

Trình tự khởi tạo chế độ 4-bit (`LCD4_INIT`)

Khi mới cấp điện, màn hình LCD luôn ở trạng thái không xác định. Trình tự khởi tạo phần mềm bằng chuỗi lệnh gửi 3 lần mã `0x30` cách nhau các khoảng trễ hợp lý giúp đưa vi mạch HD44780 về trạng thái reset đồng bộ, sau đó chuyển sang chế độ 4-bit bus (`0x20`) và cấu hình hiển thị 2 dòng ma trận 5×8 (`0x28`).

2.2.4 Thuật toán so sánh ngưỡng cảnh báo và điều khiển LED

Ngưỡng nhiệt độ cảnh báo của hệ thống được cố định tại giá trị $30°$C (`HIGH Th: 30°C`).

Khi điều khiển dàn LED trên PORT D, thách thức kỹ thuật đặt ra là các chân PD5, PD6, PD7 đang đảm nhiệm điều khiển các chân tín hiệu RW, RS, EN của màn hình LCD. Nếu ghi trực tiếp giá trị vào thanh ghi `PORTD`, trạng thái của các chân LCD sẽ bị đảo lộn, gây treo màn hình.

Để giải quyết bài toán dùng chung cổng ngoại vi này, thuật toán áp dụng các mặt nạ logic bit (Bitmask):

- Khi nhiệt độ vượt ngưỡng ($temp_int \geq 30°$C): Cần đưa 5 bit thấp (PD0 – PD4) về mức 0 (bật sáng LED), nhưng giữ nguyên giá trị của 3 bit cao (PD5 – PD7). Phép toán `AND` với mặt nạ `0xE0` (nhị phân `1110 0000`) được sử dụng:

$$\text{PORTD} = \text{PORTD} \ \& \ 0\text{xE0} \tag{2.1}$$

- Khi nhiệt độ an toàn ($temp_int < 30°$C): Cần đưa 5 bit thấp (PD0 – PD4) lên mức 1 (tắt LED), đồng thời bảo toàn 3 bit cao (PD5 – PD7). Phép toán `OR` với mặt nạ `0x1F` (nhị phân `0001 1111`) được sử dụng:

$$\text{PORTD} = \text{PORTD} \mid 0\text{x1F} \tag{2.2}$$

Nhờ kỹ thuật che mặt nạ bit này, khối cảnh báo LED và khối hiển thị LCD hoạt động hoàn toàn độc lập và tương thích tin cậy trên cùng một cổng PORT D. Chuỗi cảnh báo hiển thị trên dòng 2 của màn hình LCD có định dạng `"HIGH Th: 30°C *!"` với đúng 16 ký tự, bảo đảm không bị tràn ra ngoài bộ nhớ hiển thị (DDRAM) của màn hình LCD 1602. Chu kỳ trích mẫu giữa các lần đọc cảm biến được thiết lập là 2.0 giây (`_delay_ms(2000)`) nhằm tuân thủ khuyến nghị từ nhà sản xuất Aosong.

2.3 Quy trình biên dịch và nạp firmware

2.3.1 Quy trình tự động hóa với Makefile

Mã nguồn dự án được tổ chức và biên dịch tự động hóa thông qua công cụ GNU Make và bộ biên dịch `avr-gcc` [6]. Tập tin `Makefile` thiết lập các quy tắc dịch mã:

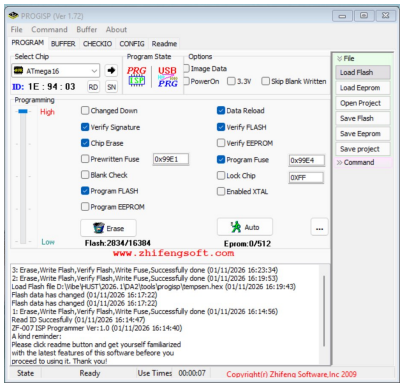
- Thiết lập cờ biên dịch: `-Wall -Os -mmcu=atmega16 -DF_CPU=8000000UL` (tối ưu mã máy `-Os`, tần số 8 MHz).
- Quy tắc tạo file `tempsen.elf` từ file nguồn `tempsen.c` và file tiêu đề `hunget_lcd.h`.
- Quy tắc chuyển đổi định dạng `tempsen.hex`: Sử dụng công cụ `avr-objcopy -O ihex -R .eeprom tempsen.elf tempsen.hex` để tạo file mã máy tiêu chuẩn Intel HEX sẵn sàng nạp vào bộ nhớ Flash ROM.

- Lệnh kiểm tra kích thước bộ nhớ: `avr-size tempsen.elf` giúp sinh viên kiểm soát chính xác tỷ lệ chiếm dụng bộ nhớ Flash và SRAM.

2.3.2 Quy trình nạp chương trình bằng phần mềm ProgISP

Các bước nạp chương trình chuẩn hóa vào vi điều khiển ATmega16 trên Kit thí nghiệm:

1. Kết nối mạch nạp USBasp giữa cổng USB của máy tính và cổng ISP 10 chân trên Kit.
2. Khởi chạy phần mềm ProgISP 1.72.
3. Tại mục Select Chip, chọn chính xác tên vi điều khiển: `ATMEGA16`.
4. Nhấn nút RD (Read) trên giao diện để kiểm tra kết nối phần cứng. Khi phần mềm đọc và báo chuỗi nhận diện `ID: 1E : 94 : 03`, mạch nạp và vi điều khiển đã thông suốt hoàn toàn.
5. Thiết lập cấu hình Fuse Bits: Nhập mã `0xD9E4` vào ô *Program Fuse* (High Fuse = `0xD9`, Low Fuse = `0xE4`). Tích chọn ô vuông kiểm tra *Program Fuse*.
6. Nạp file mã máy vào bộ đệm: Nhấn nút Load Flash, chọn đường dẫn tới file `src/tempsen.hex`. Đảm bảo thông số góc dưới bên trái thay đổi từ `Flash: 0 / 16384` sang giá trị dung lượng thực tế của file nạp.
7. Nhấn nút Auto để thực thi tuần tự các bước: Xóa chip (Erase), Ghi Flash (Write Flash), Kiểm tra lại (Verify Flash) và Ghi cầu chì (Write Fuse). Khi phần mềm báo dòng chữ *Successfully done*, vi điều khiển lập tức tự khởi động lại và bắt đầu chạy chương trình.



Hình 2.4: Giao diện phần mềm ProgISP khi nạp firmware và cấu hình Fuse Bits 0xD9E4 cho ATmega16

CHƯƠNG 3

KẾT QUẢ THỰC NGHIỆM

3.1 Kết quả thực nghiệm trên phần cứng

Hệ thống đo nhiệt độ và cảnh báo ngưỡng sử dụng vi điều khiển ATmega16 và cảm biến DHT11 đã được lắp ráp, nạp mã thực thi và kiểm thử thực tế trên Kit phát triển AVR tại phòng thí nghiệm Trường Điện – Điện tử, Đại học Bách khoa Hà Nội. Toàn bộ các trạng thái hoạt động của hệ thống được khảo sát và ghi nhận kết quả chi tiết dưới đây.

3.1.1 Giai đoạn khởi động và màn hình chào mừng

Khi vừa cấp nguồn nuôi 5V từ cổng nạp USB, vi điều khiển hoàn tất quá trình reset phần cứng, tắt cổng JTAG nội, thiết lập chiều cổng I/O và cấu hình màn hình LCD. Ngay lập tức, màn hình LCD 1602 hiển thị thông điệp chào mừng khởi động:

```
Dòng 1: DA2: DO NHIET DO
Dòng 2: DHT11 SENSOR
```

Màn hình duy trì thông điệp này trong khoảng thời gian chính xác 1.5 giây. Trong suốt giai đoạn khởi động, toàn bộ 5 đèn LED cảnh báo trên các chân PD0 – PD4 đều ở trạng thái tắt hoàn toàn nhờ lệnh khởi tạo kéo mức 1 (`PORTD |= 0x1F`). Sau 1.5 giây, lệnh xóa màn hình (`0x01`) được phát ra để chuẩn bị cho chu trình hiển thị dữ liệu đo lường.

3.1.2 Trạng thái đo lường bình thường ($T < 30°C$)

Trong điều kiện nhiệt độ phòng thí nghiệm ổn định ở mức $26.5°$C (nhỏ hơn ngưỡng cài đặt $30°$C):

- Cảm biến DHT11 thu thập dữ liệu và gửi gói tin 40-bit về MCU. Quá trình kiểm tra tính toàn vẹn dữ liệu bằng mã Checksum thành công, dữ liệu nhận được hợp lệ.
- Màn hình LCD 1602 hiển thị thông số:

```
Dòng 1: Temp: 26.5 °C
Dòng 2: HIGH Th: 30°C
```

- Do nhiệt độ đo được nằm trong dải an toàn, dòng 2 hiển thị ngưỡng `"HIGH Th: 30°C"` mà không xuất hiện bất kỳ dấu hiệu cảnh báo nào.
- Dàn 5 LED cảnh báo nối với các chân PD0 – PD4 tiếp tục được giữ ở mức logic 1, nghĩa là tắt hoàn toàn.

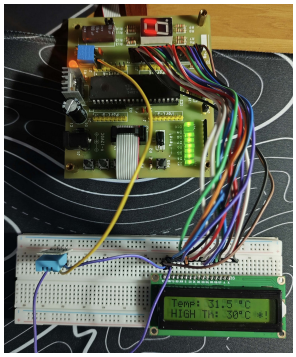
3.1.3 Trạng thái cảnh báo vượt ngưỡng quá nhiệt ($T \geq 30°C$)

Để kiểm thử phản ứng của hệ thống khi có sự cố quá nhiệt, người thực hiện sử dụng nguồn nhiệt gián tiếp (hơi ấm nhẹ áp gần bề mặt cảm biến DHT11). Khi nhiệt độ môi trường vượt qua $30.0°C$:

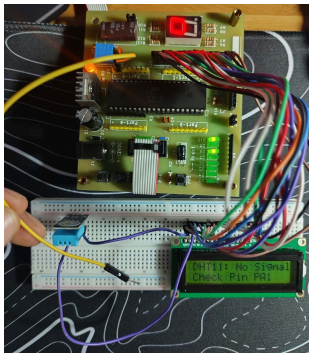
- Vi điều khiển phát hiện điều kiện $temp_int \geq 30°C$ ở chu kỳ đọc hợp lệ tiếp theo (chu kỳ lấy mẫu 2,0 giây).
- Màn hình LCD cập nhật trạng thái báo động tương ứng:

```
Dòng 1: Temp: 31.5 °C
Dòng 2: HIGH Th: 30°C *!
```

  Trên dòng 2 xuất hiện chuỗi ký hiệu báo động `"*!"` thu hút sự chú ý của người vận hành.
- Đồng thời, phép toán che mặt nạ `PORTD &= 0xE0` kéo tức thì 5 chân PD0 – PD4 xuống mức 0 (Active Low), làm cho dàn 5 đèn LED cảnh báo bật sáng đồng loạt.
- Khi nguồn nhiệt được rút ra và nhiệt độ giảm trở lại dưới $30.0°C$, màn hình tự động xóa ký hiệu `"*!"` và dàn đèn LED tự động tắt, trả hệ thống về trạng thái giám sát bình thường mà không cần thao tác reset thủ công.



(a) Màn hình LCD khi cảnh báo quá nhiệt (31.5°C)



(b) Màn hình LCD khi không có tín hiệu từ cảm biến

Hình 3.1: Hình ảnh thực tế hệ thống vận hành trong các kịch bản thử nghiệm

3.1.4 Kiểm tra phản hồi khi mất cảm biến và xử lý lỗi hệ thống

Nhằm kiểm tra độ tin cậy và khả năng tự bảo vệ của hệ thống trước các tình huống bất thường, nhóm đã tiến hành xác minh theo hai cấp độ:

- Thử nghiệm thực tế khi mất cảm biến (Kiểm chứng phần cứng): Để mô phỏng sự cố tuột dây tín hiệu, nhóm chủ động rút chân DATA (PA1). Hàm `DHT11_Read` kích hoạt bộ đếm Timeout và thoát an toàn sau tối đa 10.000 lần lặp kiểm tra của biến đếm phần mềm mà không gây treo vi điều khiển. Màn hình LCD cập nhật thông báo chẩn đoán lỗi ở lần đọc hợp lệ tiếp theo (Hình 3.1b):

```
Dòng 1: DHT11: No Signal
Dòng 2: Check Pin PA1
```

- Kiểm chứng giải thuật xử lý Checksum (Phân tích mã nguồn): Đối với trường hợp đường truyền bị can nhiễu, do chưa có thiết bị tạo xung nhiễu chuyên dụng, kịch bản sai Checksum được xác minh qua luồng logic mã nguồn (nhánh `status == -6`). Thuật toán nhận diện sự sai lệch tổng kiểm tra và hiển thị thông báo chuẩn 16 ký tự:

```
Dòng 1: DHT11: Sum Error
Dòng 2: Checksum Invalid
```

- Trạng thái xử lý lỗi xác định: Trong cả hai tình huống, vi điều khiển chủ động đưa dàn 5 LED về mức tắt (`PORTD |= 0x1F`) nhằm loại bỏ báo động giả từ dữ liệu cũ lỗi thời, đồng thời dồn sự chú ý vào thông báo trên LCD.

Bảng 3.1: Bảng tổng hợp các kịch bản kiểm tra và kết quả đánh giá hệ thống

Kịch bản kiểm tra	Phương pháp xác minh	Nhiệt độ	Nội dung hiển thị trên LCD 1602	Trạng thái dàn LED
Khởi động hệ thống	Thực nghiệm trên Kit phần cứng	–	Dòng 1: `DA2: DO NHIET DO` Dòng 2: `DHT11 SENSOR`	Tắt toàn bộ
Nhiệt độ bình thường	Thực nghiệm trên Kit phần cứng	26.5°C	Dòng 1: `Temp: 26.5 °C` Dòng 2: `HIGH Th: 30°C`	Tắt toàn bộ
Nhiệt độ vượt ngưỡng	Thực nghiệm trên Kit phần cứng	31.5°C	Dòng 1: `Temp: 31.5 °C` Dòng 2: `HIGH Th: 30°C *!`	Bật sáng cả 5 LED
Mất tín hiệu cảm biến	Thực nghiệm trên Kit (Rút chân PA1)	Không xác định	Dòng 1: `DHT11: No Signal` Dòng 2: `Check Pin PA1`	Tắt toàn bộ (Xác định)
Sai mã Checksum	Phân tích giải thuật mã nguồn	Không xác định	Dòng 1: `DHT11: Sum Error` Dòng 2: `Checksum Invalid`	Tắt toàn bộ (Xác định)

3.2 Đánh giá kết quả thực nghiệm

Qua quá trình thử nghiệm thực tế trên Kit phần cứng, hệ thống đã đạt các kết quả đề ra:

- Độ ổn định và tính hợp lệ dữ liệu: DHT11 và ATmega16 giao tiếp ổn định qua bus 1-Wire chu kỳ 2.0 giây. Thuật toán Checksum cộng 8-bit giúp phát hiện một số lỗi dữ liệu truyền do biến dạng tín hiệu hoặc xung nhiễu đường truyền, dù không bảo đảm phát hiện được mọi tổ hợp lỗi bit đồng thời.
- Khả năng hiển thị và minh chứng thực tế: Kết quả thử nghiệm trên Kit (Hình 3.1a) cho thấy màn hình LCD ghi nhận giá trị 31.5°C từ chương trình đọc cảm biến, đồng thời hiển thị ký hiệu cảnh báo `"*!"` và kích hoạt dàn 5 LED sáng rõ nét. Mặc dù chuẩn tài liệu cơ bản DHT11 thường chỉ định byte thập phân bằng 0, giá trị 31.5°C phản ánh sự giải mã trực tiếp của vi điều khiển từ các byte dữ liệu nhận được trong thử nghiệm.
- Trạng thái xử lý lỗi xác định: Khi mất cảm biến hoặc sai Checksum, cơ chế Timeout chống treo vi điều khiển, chủ động đưa dàn 5 LED về trạng thái tắt xác định (`PORTD |= 0x1F`) và hiển thị thông điệp chẩn đoán chi tiết trên LCD để hỗ trợ người vận hành.

CHƯƠNG 4

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết luận

Trải qua quá trình nghiên cứu lý thuyết, thiết kế sơ đồ, lập trình phần mềm nhúng và kiểm nghiệm thực tế trên Kit phát triển, đề tài "Thiết kế hệ thống đo nhiệt độ và cảnh báo ngưỡng sử dụng ATmega16" thuộc học phần Đồ án Thiết kế II đã hoàn thành đầy đủ và toàn diện các mục tiêu kỹ thuật đề ra ban đầu:

1. Về kiến thức và kỹ năng chuyên môn:
    - Nắm vững sâu sắc kiến trúc vi điều khiển AVR RISC 8-bit (cụ thể là vi điều khiển ATmega16), hiểu rõ cấu trúc các thanh ghi chức năng đặc biệt (`DDRx`, `PORTx`, `PINx`, `MCUCSR`) và thành thạo kỹ thuật lập trình C nhúng trực tiếp trên thanh ghi (Bare-metal programming) với bộ công cụ chuẩn GNU `avr-gcc`.
    - Làm chủ hoàn toàn giao thức truyền thông đơn tuyến 1-Wire Single-Bus với cảm biến số DHT11: từ việc phát xung kích hoạt Start chính xác đến từng micro giây, giải mã xung nhị phân theo độ rộng thời gian (Pulse Width Modulation) đến việc kiểm tra toàn vẹn gói tin bằng thuật toán Checksum 8-bit.
    - Xây dựng và hoàn thiện thư viện điều khiển màn hình tinh thể lỏng LCD 1602 ở chế độ 4-bit bus (`hunget_lcd.h`), giải quyết thành công bài toán chia sẻ và che mặt nạ bit (Bitmasking) trên cổng ngoại vi PORT D dùng chung giữa đường điều khiển LCD và dàn LED cảnh báo.
    - Làm chủ quy trình nạp chương trình, thiết lập cầu chì Fuse Bits (`0xD9E4`), xử lý triệt để bài toán xung đột phần cứng JTAG và tối ưu hóa hệ thống vi điều khiển.
2. Về sản phẩm và ứng dụng thực tiễn:
    - Xây dựng thành công một hệ thống đo nhiệt độ thời gian thực hoạt động bền bỉ, độ tin cậy cao, hiển thị trực quan thông số nhiệt độ kèm phần thập phân và ngưỡng giám sát trên màn hình LCD 1602.

- Cơ chế cảnh báo ngưỡng quá nhiệt ($30°C$) vận hành nhanh nhạy, đồng thời có khả năng tự phục hồi khi nhiệt độ trở lại bình thường và thông báo lỗi rõ ràng khi có sự cố đứt kết nối cảm biến.

4.2 Hạn chế của đề tài

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kỹ thuật cơ bản, đồ án vẫn còn một số điểm hạn chế do giới hạn về mặt thời gian và trang thiết bị thực nghiệm:

- Cảm biến DHT11 có dải đo nhiệt độ hẹp ($0°C - 50°C$) và sai số tương đối lớn ($\pm 2.0°C$), chu kỳ lấy mẫu tương đối chậm (khuyến nghị 2.0s mỗi lần đo theo tài liệu Aosong) nên chưa phù hợp cho các môi trường đòi hỏi đáp ứng thời gian thực cực nhanh.
- Ngưỡng cảnh báo nhiệt độ ($30°C$) hiện đang được cố định cứng trong mã nguồn (Hard-coded), người sử dụng chưa thể linh hoạt thay đổi ngưỡng trực tiếp từ bên ngoài bằng phím bấm mà phải biên dịch lại chương trình.
- Cơ chế xử lý khi mất cảm biến hiện đưa toàn bộ LED về trạng thái tắt để tránh báo động giả từ dữ liệu cũ lỗi thời, nhưng có nhược điểm là không tiếp tục cảnh báo bằng đèn nếu môi trường thực tế vẫn đang quá nhiệt.
- Hệ thống chưa có bộ nhớ lưu trữ lịch sử biến thiên nhiệt độ theo thời gian và chưa có khả năng truyền thông dữ liệu từ xa lên mạng máy tính.

4.3 Hướng phát triển mở rộng

Từ nền tảng phần cứng và phần mềm đã được kiểm chứng của đồ án, hệ thống có thể tiếp tục được nghiên cứu và nâng cấp theo các hướng ứng dụng thực tế:

1. Nâng cấp cảm biến công nghiệp: Thay thế cảm biến DHT11 bằng các dòng cảm biến kỹ thuật số có độ chính xác cao hơn như DHT22 (sai số $\pm 0.5°C$, dải đo $-40°C$ đến $80°C$) hoặc các dòng cảm biến giao tiếp I2C cao cấp như Sensirion SHT30/SHT35 hoặc Bosch BME280 (đo đồng thời nhiệt độ, độ ẩm và áp suất khí quyển).
2. Bổ sung giao diện người dùng tương tác linh hoạt: Tích hợp bàn phím ma trận 4×4 hoặc cụm nút bấm điều hướng (Up/Down/Set) kết hợp còi báo động Buzzer,

cho phép người dùng tự do cài đặt ngưỡng nhiệt độ cảnh báo và lưu trữ giá trị cấu hình này vào bộ nhớ không biến động EEPROM của ATmega16.

3. Bổ sung chế độ cảnh báo lỗi cảm biến chuyên biệt: Lập trình cơ chế nhấp nháy toàn bộ dàn LED (tần số 2 Hz) hoặc còi buzzer ngắt quãng khi mất kết nối cảm biến để người vận hành phân biệt rõ ràng giữa trạng thái ”Bình thường” (LED tắt), ”Cảnh báo quá nhiệt” (LED sáng liên tục) và ”Lỗi cảm biến” (LED nhấp nháy).

4. Tích hợp kết nối không dây và phát triển ứng dụng IoT: Kết nối cổng giao tiếp nối tiếp USART của ATmega16 với các mô-đun truyền thông không dây như Wi-Fi ESP8266/ESP32, Bluetooth HC-05 hoặc chuẩn truyền xa LoRa để gửi dữ liệu nhiệt độ lên đám mây (Cloud Server) qua giao thức MQTT/HTTP. Qua đó, người quản lý có thể giám sát biểu đồ nhiệt độ môi trường và nhận thông báo cảnh báo khẩn cấp trực tiếp trên ứng dụng điện thoại thông minh (Blynk, ThingsBoard) ở bất kỳ đâu.

TÀI LIỆU THAM KHẢO

[1] Atmel Corporation, “ATmega16/ATmega16L 8-bit AVR Microcontroller with 16K Bytes In-System Programmable Flash Datasheet”, Document Rev. 2466T–AVR–07/10, San Jose, CA, USA, 2010.

[2] Aosong Electronics Co., Ltd, “DHT11 Humidity & Temperature Sensor Product Manual”, Guangzhou Aosong Electronics Co., Ltd, Guangzhou, China, 2015.

[3] Hitachi Semiconductor and Integrated Circuits, “HD44780U (Dot Matrix Liquid Crystal Display Controller/Driver) Technical Data”, Rev. 0.0, Tokyo, Japan, 1998.

[4] Trường Điện – Điện tử (Viện Điện tử – Viễn thông), Đại học Bách khoa Hà Nội, “Tài liệu hướng dẫn thực hành và sơ đồ nguyên lý Kit vi điều khiển AVR ATmega16”, Hà Nội, 2022.

[5] Ngô Diên Tập, “Kỹ thuật vi điều khiển và ứng dụng – Lập trình họ vi điều khiển AVR”, Nhà xuất bản Khoa học và Kỹ thuật, Hà Nội, 2012.

[6] Free Software Foundation, “AVR-GCC and AVR Libc User Manual – A C runtime library for Atmel AVR microcontrollers”, https://www.nongnu.org/avr-libc/, truy cập tháng 10/2026.

[7] Trường Điện – Điện tử, Đại học Bách khoa Hà Nội, “Quy định về thể thức văn bản và cấu trúc báo cáo Đồ án môn học – Đồ án tốt nghiệp bậc đại học”, Hà Nội, 2023.

PHỤ LỤC: TOÀN BỘ MÃ NGUỒN CHƯƠNG TRÌNH

Phụ lục A: File mã nguồn chính `tempsen.c`

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include "hunget_lcd.h" // Thư viện LCD chuẩn Kit BK

// ======================================================
// CẤU HÌNH CHÂN DHT11 NỐI VÀO CHÂN PA1 TRÊN PORT A
// ======================================================
#define DHT_DDR   DDRA
#define DHT_PORT  PORTA
#define DHT_PIN_R PINA
#define DHT_BIT   PA1

// Hàm đọc dữ liệu từ cảm biến số DHT11
// Trả về 0 nếu thành công, < 0 nếu lỗi/timeout
int8_t DHT11_Read(uint8_t *temp_int, uint8_t *temp_dec, uint8_t *humi_int) {
    uint8_t data[5] = {0, 0, 0, 0, 0};
    uint16_t timeout;

    // 1. MCU gửi tín hiệu Start (kéo LOW tối thiểu 18ms)
    DHT_DDR |= (1 << DHT_BIT);     // Set PA1 là Output
    DHT_PORT &= ~(1 << DHT_BIT);   // Kéo xuống LOW
    _delay_ms(20);                 // Giữ mức LOW 20ms

    // 2. Nhả bus sang Input có trở kéo lên (tránh xung đột)
    DHT_DDR &= ~(1 << DHT_BIT);    // Chuyển PA1 thành Input
    DHT_PORT |= (1 << DHT_BIT);    // Bật điện trở kéo lên nội
    _delay_us(30);                 // Chờ 20-40us cho DHT11 bắt đầu phản hồi

    // Đợi DHT11 kéo LOW (khoảng 20-40µs)
    timeout = 10000;
    while (DHT_PIN_R & (1 << DHT_BIT)) {
        if (--timeout == 0) return -1; // Timeout chờ DHT11 phản hồi
    }

    // Đợi DHT11 kéo HIGH (khoảng 80µs)
    timeout = 10000;
```

```
while (!(DHT_PIN_R & (1 << DHT_BIT))) {
    if (--timeout == 0) return -2;
}

// Đợi DHT11 kéo LOW chuẩn bị truyền bit đầu tiên (khoảng 80µs)
timeout = 10000;
while (DHT_PIN_R & (1 << DHT_BIT)) {
    if (--timeout == 0) return -3;
}

// 3. Đọc 40 bit dữ liệu (5 byte)
for (uint8_t i = 0; i < 5; i++) {
    for (int8_t j = 7; j >= 0; j--) {
        // Đợi hết xung LOW 50µs của từng bit
        timeout = 10000;
        while (!(DHT_PIN_R & (1 << DHT_BIT))) {
            if (--timeout == 0) return -4;
        }

        // Đo độ rộng xung HIGH:
        // 26-28µs -> bit 0
        // 70µs    -> bit 1
        // Chờ 38µs rồi đọc trạng thái chân
        _delay_us(38);
        if (DHT_PIN_R & (1 << DHT_BIT)) {
            data[i] |= (1 << j);
            // Đợi xung HIGH kết thúc
            timeout = 10000;
            while (DHT_PIN_R & (1 << DHT_BIT)) {
                if (--timeout == 0) return -5;
            }
        }
    }
}

// 4. Kiểm tra mã kiểm tra lỗi (Checksum)
if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4]) {
    return -6; // Sai checksum
}

*humi_int  = data[0];
*temp_int  = data[2];
*temp_dec  = data[3];
```

```
    return 0; // Đọc thành công
}

int main(void) {
    // 1. TẮT JTAG (BẮT BUỘC để giải phóng PORT C điều khiển LCD)
    MCUCSR = (1 << JTD);
    MCUCSR = (1 << JTD);

    // 2. KHỞI TẠO LCD
    LCD4_INIT(0, 0);

    // 3. KHỞI TẠO DÃY LED CẢNH BÁO (PD0 - PD4)
    // PD5-PD7 là Output cho LCD (đã set trong thư viện LCD)
    // Set thêm PD0-PD4 là Output cho LED
    DDRD |= 0x1F;  // Set 5 chân thấp là Output
    PORTD |= 0x1F; // Xuất mức 1 để TẮT hết LED ban đầu (Active Low)

    // --- MÀN HÌNH CHÀO MỪNG ---
    LCD4_CUR_GOTO(1, 0);
    LCD4_OUT_STR("DA2: DO NHIET DO");
    LCD4_CUR_GOTO(2, 2);
    LCD4_OUT_STR("DHT11 SENSOR");
    _delay_ms(1500);
    LCD4_OUT_CMD(0x01); // Xóa màn hình
    _delay_ms(2);

    char buffer[32];
    uint8_t temp_int = 0;
    uint8_t temp_dec = 0;
    uint8_t humi_int = 0;

    while (1) {
        int8_t status = DHT11_Read(&temp_int, &temp_dec, &humi_int);

        if (status == 0) {
            // --- ĐỌC THÀNH CÔNG TỪ DHT11 ---
            // Dòng 1: "Temp: 31.5 °C" (16 ký tự chuẩn)
            LCD4_CUR_GOTO(1, 0);
            sprintf(buffer, "Temp: %u.%u \xDF""C   ", temp_int, temp_dec);
            LCD4_OUT_STR(buffer);

            // Dòng 2: Hiển thị ngưỡng cảnh báo "HIGH Th: 30°C"
            LCD4_CUR_GOTO(2, 0);
```

```
            if (temp_int >= 30) {
                // Nhiệt độ >= 30 C: Báo WARNING và BẬT LED (16 ký tự)
                LCD4_OUT_STR("HIGH Th: 30\xDF""C *!");
                PORTD &= 0xE0; // Bật sáng 5 LED cảnh báo (Active LOW)
            } else {
                // Nhiệt độ bình thường <= 30 C: TẮT LED (16 ký tự)
                LCD4_OUT_STR("HIGH Th: 30\xDF""C   ");
                PORTD |= 0x1F; // Tắt 5 LED cảnh báo
            }
        } else if (status == -6) {
            // Lỗi kiểm tra toàn vẹn dữ liệu (Checksum Error)
            LCD4_CUR_GOTO(1, 0);
            LCD4_OUT_STR("DHT11: Sum Error");
            LCD4_CUR_GOTO(2, 0);
            LCD4_OUT_STR("Checksum Invalid");
            PORTD |= 0x1F; // Trạng thái xác định: Tắt LED cảnh báo
        } else {
            // Trường hợp mất tín hiệu, chưa cắm chân PA1 hoặc lỏng dây
            LCD4_CUR_GOTO(1, 0);
            LCD4_OUT_STR("DHT11: No Signal");
            LCD4_CUR_GOTO(2, 0);
            LCD4_OUT_STR("Check Pin PA1   ");
            PORTD |= 0x1F; // Trạng thái xác định: Tắt LED cảnh báo
        }

        // Chu kỳ lấy mẫu khuyến nghị theo datasheet của Aosong (2 giây)
        _delay_ms(2000);
    }

    return 0;
}
```

Phụ lục B: Thư viện điều khiển LCD `hunget_lcd.h`

```
#ifndef HUNGET_LCD_H
#define HUNGET_LCD_H

#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include <stdlib.h>

// ======================================================
// CẤU HÌNH CHÂN THEO ĐÚNG SƠ ĐỒ MẠCH KIT
// ======================================================
#define LCD_DATA_DDR  DDRC
#define LCD_DATA_PORT PORTC
#define LCD_DATA_PIN  PINC

#define LCD_CTRL_DDR  DDRD
#define LCD_CTRL_PORT PORTD

#define LCD_RW 5 // Pin 5 LCD nối PD5
#define LCD_RS 6 // Pin 4 LCD nối PD6
#define LCD_EN 7 // Pin 6 LCD nối PD7

static inline void LCD_Enable(void) {
    LCD_CTRL_PORT |= (1 << LCD_EN);
    _delay_us(5);
    LCD_CTRL_PORT &= ~(1 << LCD_EN);
    _delay_us(100);
}

static inline void LCD_Send4Bit(unsigned char Data) {
    unsigned char current_port_c = LCD_DATA_PORT & 0x0F;
    LCD_DATA_PORT = current_port_c | (Data & 0xF0);
    LCD_Enable();
}

static inline void LCD4_OUT_CMD(unsigned char command) {
    LCD_CTRL_PORT &= ~(1 << LCD_RS);
    LCD_CTRL_PORT &= ~(1 << LCD_RW);
    LCD_Send4Bit(command);
    LCD_Send4Bit(command << 4);
    if (command < 4) _delay_ms(2);
```

```
    else _delay_us(50);
}

static inline void LCD4_OUT_DATA(unsigned char data) {
    LCD_CTRL_PORT |= (1 << LCD_RS);
    LCD_CTRL_PORT &= ~(1 << LCD_RW);
    LCD_Send4Bit(data);
    LCD_Send4Bit(data << 4);
    _delay_us(50);
}

static inline void LCD4_INIT(unsigned char cursor, unsigned char blink) {
    LCD_DATA_DDR |= 0xF0;
    LCD_CTRL_DDR |= (1<<LCD_RS) | (1<<LCD_RW) | (1<<LCD_EN);
    LCD_CTRL_PORT &= ~((1<<LCD_RS) | (1<<LCD_RW) | (1<<LCD_EN));
    _delay_ms(50);

    LCD_Send4Bit(0x30); _delay_ms(5);
    LCD_Send4Bit(0x30); _delay_ms(2);
    LCD_Send4Bit(0x30); _delay_ms(2);

    LCD_Send4Bit(0x20); _delay_ms(2);

    LCD4_OUT_CMD(0x28); _delay_ms(1);
    LCD4_OUT_CMD(0x08); _delay_ms(1);
    LCD4_OUT_CMD(0x01); _delay_ms(3);

    unsigned char cmd = 0x0C;
    if(cursor) cmd |= 0x02;
    if(blink) cmd |= 0x01;
    LCD4_OUT_CMD(cmd);
    _delay_ms(1);
    LCD4_OUT_CMD(0x06);
    _delay_ms(1);
}

void LCD4_CUR_GOTO(unsigned char row, unsigned char col) {
    unsigned char address;
    if (row == 1) address = 0x80 + col;
    else address = 0xC0 + col;
    LCD4_OUT_CMD(address);
}
```

```
void LCD4_OUT_STR(char *str) {
    while (*str) LCD4_OUT_DATA(*str++);
}

void LCD4_OUT_DEC(unsigned int num, unsigned char len) {
    char buffer[10];
    sprintf(buffer, "%05u", num);
    char *ptr = &buffer[5 - len];
    LCD4_OUT_STR(ptr);
}

void LCD4_DIS_SHIFT(unsigned char direct, unsigned char step) {
    unsigned char i;
    if (direct == 0)
        for(i=0; i<step; i++) LCD4_OUT_CMD(0x18);
    else
        for(i=0; i<step; i++) LCD4_OUT_CMD(0x1C);
}

#endif // HUNGET_LCD_H
```